Q2 - When Does Short-Circuit Evaluation Matter? (4 marks)

2.1 Experimental design

Concession rule: (senior && class_ok) || disability_card, where verify_card() is the slow server call and is the only function whose calls are counted. 1,000 sample passengers were generated deterministically: 250 seniors (25%), 334 with an eligible class, 84 who are both senior and eligible, and 50 disability-card holders. Experiment 1 uses functions that print when evaluated; Experiment 2 counts the calls to verify_card() for different orderings and operators; Experiment 3 repeats it with an audit-logging verifier.

q2.c

```
#include <stdio.h>

#define N 1000
static int calls = 0;           /* number of calls to the slow function */
static int audit_log = 0;       /* passengers recorded in the audit log */

typedef struct { int senior, class_ok, card; } Passenger;
static Passenger p[N];

/* Slow verification of the disability card (server call). */
static int verify_card(const Passenger *x) { calls++; return x->card; }
/* Changed condition: same call but also writes an audit-log record. */
static int verify_card_audit(const Passenger *x) { calls++; audit_log++; return x->card; }

static int A1(void) { int g = 0; for (int i=0;i<N;i++) if ((p[i].senior && p[i].class_ok) || verify_card(&p[i])) g++; return g; }
static int A2(void) { int g = 0; for (int i=0;i<N;i++) if (verify_card(&p[i]) || (p[i].senior && p[i].class_ok)) g++; return g; }
static int A3(void) { int g = 0; for (int i=0;i<N;i++) if ((p[i].class_ok && p[i].senior) || verify_card(&p[i])) g++; return g; }
static int A4(void) { int g = 0; for (int i=0;i<N;i++) if (((p[i].senior && p[i].class_ok) | verify_card(&p[i]))) g++; return g; }
static int A5(void) { int g = 0; for (int i=0;i<N;i++) { if (p[i].senior) { if (p[i].class_ok) { g++; continue; } } if (verify_card(&p[i])) g++; }
return g; }
static int A6(void) { int g = 0; for (int i=0;i<N;i++) if (((p[i].senior & p[i].class_ok) | verify_card(&p[i]))) g++; return g; }
static int A7(void) { int g = 0; for (int i=0;i<N;i++) { int c = verify_card_audit(&p[i]); if (c || (p[i].senior && p[i].class_ok)) g++; } return
g; }
static int A8(void) { int g = 0; for (int i=0;i<N;i++) if ((p[i].senior && p[i].class_ok) || verify_card_audit(&p[i])) g++; return g; }

static int f(int v, const char *n) { printf("   %s evaluated\n", n); return v; }

int main(void)
{
   /* Sample data: 25% senior, 33% eligible class, 5% hold a disability card */
   int s=0, e=0, c=0, both=0;
   for (int i = 0; i < N; i++) {
      p[i].senior   = (i % 4 == 0);
      p[i].class_ok = (i % 3 == 0);
      p[i].card     = (i % 20 == 7);
      s += p[i].senior; e += p[i].class_ok; c += p[i].card; both += p[i].senior && p[i].class_ok;
   }
   printf("Sample: %d passengers, %d seniors, %d eligible class, %d senior&&eligible, %d card holders\n\n", N, s, e, both, c);

   printf("Experiment 1: when is the 2nd operand skipped?\n");
   printf(" f(0,\"A\") && f(1,\"B\"):\n"); int r1 = f(0,"A") && f(1,"B"); printf("   result=%d\n", r1);
   printf(" f(1,\"A\") && f(1,\"B\"):\n"); int r2 = f(1,"A") && f(1,"B"); printf("   result=%d\n", r2);
   printf(" f(1,\"A\") || f(1,\"B\"):\n"); int r3 = f(1,"A") || f(1,"B"); printf("   result=%d\n", r3);
   printf(" f(0,\"A\") || f(1,\"B\"):\n"); int r4 = f(0,"A") || f(1,"B"); printf("   result=%d\n", r4);
   printf(" f(0,\"A\") &  f(1,\"B\"):\n"); int r5 = f(0,"A") &  f(1,"B"); printf("   result=%d\n", r5);
   printf(" f(1,\"A\") |  f(1,\"B\"):\n"); int r6 = f(1,"A") |  f(1,"B"); printf("   result=%d\n", r6);
   printf(" 2 && 1 = %d,  2 & 1 = %d\n\n", 2 && 1, 2 & 1);

   struct { const char *name; int (*fn)(void); } t[] = {
      {"V1 (senior&&class) || card()          ", A1},
      {"V2 card() || (senior&&class)          ", A2},
      {"V3 (class&&senior) || card()          ", A3},
      {"V4 nested if, card() last             ", A5},
      {"V5 (senior&&class) | card()  [bitwise]", A4},
      {"V6 (senior&class) | card()   [bitwise]", A6},
   };
   printf("Experiment 2: calls to slow verify_card() for N=%d\n", N);
```

```
    for (int k = 0; k < 6; k++) { calls = 0; int g = t[k].fn(); printf(" %s calls=%4d  concessions=%d\n", t[k].name, calls, g); }

    printf("\nChanged condition (audit log, every passenger must be recorded):\n");
    calls = 0; audit_log = 0; int g = A8(); printf(" short-circuit order V1  : calls=%d audit_records=%d concessions=%d\n", calls, audit_log,
g);
    calls = 0; audit_log = 0; g = A7();     printf(" call first, then test   : calls=%d audit_records=%d concessions=%d\n", calls, audit_log, g);
    return 0;
}
```

Observed output

```
Sample: 1000 passengers, 250 seniors, 334 eligible class, 84 senior&&eligible, 50 card holders

Experiment 1: when is the 2nd operand skipped?
 f(0,"A") && f(1,"B"):
   A evaluated
   result=0
 f(1,"A") && f(1,"B"):
   A evaluated
   B evaluated
   result=1
 f(1,"A") || f(1,"B"):
   A evaluated
   result=1
 f(0,"A") || f(1,"B"):
   A evaluated
   B evaluated
   result=1
 f(0,"A") &  f(1,"B"):
   A evaluated
   B evaluated
   result=0
 f(1,"A") |  f(1,"B"):
   A evaluated
   B evaluated
   result=1
 2 && 1 = 1,  2 & 1 = 0

Experiment 2: calls to slow verify_card() for N=1000
 V1 (senior&&class) || card()          calls= 916  concessions=134
 V2 card() || (senior&&class)          calls=1000  concessions=134
 V3 (class&&senior) || card()          calls= 916  concessions=134
 V4 nested if, card() last             calls= 916  concessions=134
 V5 (senior&&class) | card()  [bitwise] calls=1000  concessions=134
 V6 (senior&class) | card()   [bitwise] calls=1000  concessions=134

Changed condition (audit log, every passenger must be recorded):
 short-circuit order V1  : calls=916 audit_records=916 concessions=134
 call first, then test   : calls=1000 audit_records=1000 concessions=134
```

Programiz
C Online Compiler
Client wants more options? Done. Pay via UPI
Buy now
Adobe Creative Cloud Pro
Programiz PRO

main.c

```
#include <stdio.h>

#define N 1000
static int calls = 0;             /* number of calls to the slow function */
static int audit_log = 0;         /* passengers recorded in the audit log */

typedef struct { int senior, class_ok, card; } Passenger;
static Passenger p[N];

/* Slow verification of the disability card (server call). */
static int verify_card(const Passenger *x) { calls++; return x->card; }
/* Changed condition: same call but also writes an audit-log record. */
```

Clear | Run ▶

Output

```
Sample: 1000 passengers, 250 seniors, 334 eligible class, 84 senior&&eligible, 50 card holders

Experiment 1: when is the 2nd operand skipped?
 f(0,"A") && f(1,"B"):
   A evaluated
   result=0
 f(1,"A") && f(1,"B"):
   A evaluated
   B evaluated
   result=1
 f(1,"A") || f(1,"B"):
   A evaluated
   result=1
 f(0,"A") || f(1,"B"):
```

2.2 Results and explanation

When is the second operand not evaluated? For A && B, B is skipped when A is 0 (false), because the result must be 0. For A || B, B is skipped when A is non-zero (true), because the result must be 1. In the output, f(0,"A") && f(1,"B") and f(1,"A") || f(1,"B") never evaluated B. C also guarantees left-to-right evaluation with a sequence point after the first operand, so p != NULL && p->x > 0 is safe.

Version	Calls to slow function	Concessions	Calls saved
V1 (senior && class_ok) || card()	916	134	84
V2 card() || (senior && class_ok)	1000	134	0
V3 (class_ok && senior) || card()	916	134	84
V4 nested if, card() checked last	916	134	84
V5 (senior && class_ok) | card()	1000	134	0
V6 (senior & class_ok) | card()	1000	134	0

Effect of order. Putting the slow check last (V1) skips it for the 84 passengers who already qualify through senior && class_ok (916 calls); putting it first (V2) calls it for all 1,000. All versions give the same 134 concessions - only the cost differs, because the operands have no side effects.

Difference between `&` and `&&`. && is a logical operator: it short-circuits and gives 0 or 1. & is a bitwise operator: it always evaluates both operands (V5/V6 made 1,000 calls) and the order in which they are evaluated is unspecified. & also works on bit patterns, not truth values: 2 && 1 is 1 but 2 & 1 is 0 (observed), so using & on conditions that are "non-zero true" but not exactly 1 gives wrong answers.

When should the cheapest or most-likely-false condition go first? For &&, write first the condition that is cheap and most likely to be false, because a false first operand skips the rest. For ||, write first the cheap condition that is most likely true. The expected cost of A && B is cost(A) + P(A true) x cost(B), so the expensive condition should come last and be reached rarely. Here the slow card check belongs last in the ||. Within senior && class_ok, the rarer condition (senior, 25% versus 33%) goes first.

When is the order irrelevant? When both conditions are cheap, have no side effects, and neither guards the other (V1 and V3 gave identical call counts and results). But order is NOT irrelevant when the left operand protects the right one (i < n && a[i] > 0, p != NULL && p->x), because the right side must not be evaluated if the guard fails.

2.3 Changed condition - the verifier writes an audit log

Now verify_card() has a side effect, so short-circuiting changes what the program does, not only how fast it runs. Observed: with the cheap-first order V1 only 916 passengers were checked and logged, leaving **84 passengers missing from the audit log**; calling the verifier first for everybody made 1,000 calls and 1,000 audit records, with the same 134 concessions.

- If the audit must contain **every** passenger, the verifier has to run unconditionally. The recommended order reverses: call it first, store the result, then combine: int card_ok = verify_card_audit(p); if (card_ok || (senior && class_ok)) .... Avoid hiding this behind | - an explicit statement documents the intent, and | leaves the operand order unspecified.
- If the log should record only passengers who were actually verified against the server, the cheap-first order V1 is still correct (the log is consistent with the calls made), and it saves 84 server calls.

The key lesson: once an operand has side effects (logging, counters, I/O), cost-based reordering is no longer a free optimisation; correctness requirements decide the order.

2.4 Alternative concept and complexity analysis

Let n be the number of passengers and k the number for whom senior && class_ok is already true (k = 84 in the sample). Calls to the slow function:

Version	Calls (time)	Best / worst	Notes
Short-circuit (S && C) || card()	n - k	0 / n	Minimum calls; compact
Nested if statements (card checked last)	n - k	0 / n	Same behaviour and calls as &&/||, but longer code
Non-short-circuit (S & C) | card()	n (always)	n / n	Every passenger calls the server; operand order unspecified; wrong for non-0/1 values
Slow function first card() || (S && C)	n (always)	n / n	Needed only when an audit log must be complete

All versions are O(n) in the worst case and O(1) extra space; they differ in the constant factor, which matters a lot when each call is a network round-trip. Nested ifs and &&/|| are equivalent in cost (compilers compile && into the same conditional jumps), so the choice between them is about readability; the &/| version is the only one that loses the saving.